

Autonomous Research Agent for BirdCLEF+ 2026

Danish Mujtaba Qureshi, Lorena Alexandra Leicht, Sebastian Mis, Maximilian Noelle-Wying

Católica Lisbon - MSc Business Analytics - Advanced Predictive Analytics

May 2026

Abstract

We built an autonomous research agent that designs, generates, executes, and judges its own deep-learning experiments for the BirdCLEF+ 2026 Kaggle competition (Track B, 234-class multi-label bird species recognition from mel-spectrograms). A locally-hosted LLM proposes architectures, writes a `build_model` block inside a fixed Jinja2 training skeleton, and a separate judge call decides whether each experiment improves on memory. Over 20 studies and 61 experiments, the agent reached Kaggle public-LB ROC-AUC **0.827** and private-LB **0.810** (best of 4 manual submissions; two public, two private); internal validation reaches 0.998 but we document a 17-point val/leaderboard gap consistent with recorder leakage in the val split. The key design choice is to constrain LLM codegen to a single `build_model` block: this bounds the failure surface and makes experiments comparable, at the cost of the agent never proposing novel training tricks.

Video presentation (D3): *TODO - to be replaced with the unlisted YouTube URL before submission. Source: videos/demo.mp4 in the repository.*

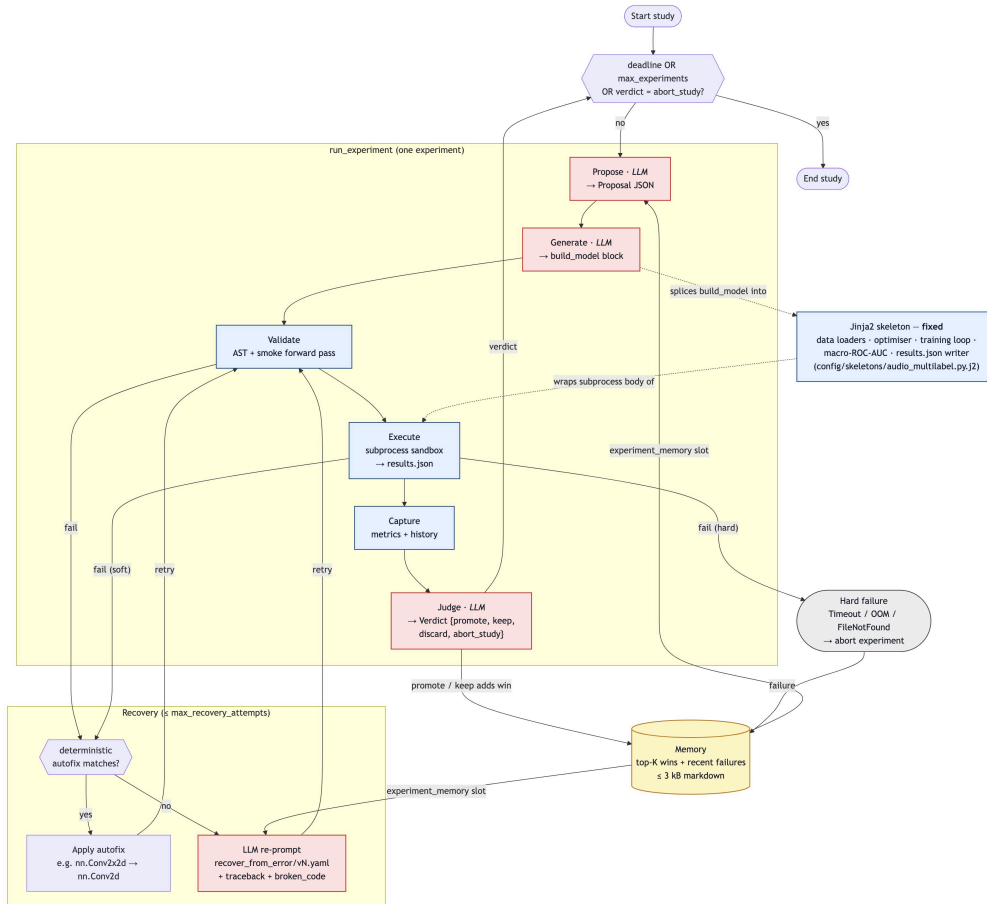


Figure 1. Autonomous research agent loop. Outer study iterates experiments under a deadline + max-experiments budget; each experiment runs the inner pipeline Propose → Generate → Validate → Execute → Capture → Judge. Memory (top-K wins + recent failures) feeds Propose and the recovery re-prompt. Recovery activates on Validate or soft Execute failures (autofix, otherwise LLM re-prompt). The Jinja2 skeleton fixes everything outside build_model.

1 Agent architecture

The agent runs a single closed loop: an outer **study** iterates **experiments**, each of which moves through a fixed seven-stage pipeline: *Propose*, *Generate*, *Validate*, *Execute*, *Capture*, *Judge*, plus an opportunistic *Recover* branch on failure (Figure 1). The LLM owns three stages (Propose, Generate, Judge) and is invoked again only when Recover fires; the training loop, the data loaders, the optimiser, metric computation, and result serialisation are fixed Python that the LLM never edits. The outer loop in `lab/core/lifecycle.py:StudyRunner.run` stops when one of three conditions trips: the wallclock deadline (`max_wallclock_minutes`), the experiment cap (`max_experiments`), or a per-experiment `abort_study` verdict from the judge. The inner pipeline lives in `lab/core/experiment.py:run_experiment`.

LLM interface. Every call to a model goes through `LLMClient.chat(messages) → str` in `lab/core/llm.py`. One method, three back-ends: a default Ollama endpoint hosting different models on the host machine. The implementation uses only `urllib` from the standard library, so swapping providers is a YAML change rather than a dependency change. Transient errors (HTTP 429, 5xx, timeouts) raise `LLMTransientError` and are retried with exponential backoff up to `retry_attempts`; permanent errors (4xx other than 429, malformed JSON, unknown provider) raise `LLMPermanentError` and fail immediately. Each successful call snapshots token counts and round-trip time so the dashboard can attribute cost per stage.

Prompt management. Every prompt is a versioned YAML file under `config/prompts/<task>/vN.yaml` (six tasks: `propose`, `generate`, `recover`, `judge-experiment`, `judge-study`, `analyze`), indexed by `config/prompts/_registry.yaml`, which maps each task to a single active version. The pointer file is the only mutable surface; the version files themselves are immutable once pinned (ADR-004). To change a prompt’s wording the contributor writes a new `vN+1.yaml` and bumps the registry - old versions stay on disk so historical studies remain reproducible. `PromptEngine` fills task-specific slots at render time (task description, class count, input shape, EDA summary, experiment memory, personality, valid families, code skeleton for `propose`; the rendered proposal plus the skeleton for `codegen`; broken code, error type, and traceback for `recover`). Every study

records the exact prompt paths it used in `Study.prompt_template_paths`.

Memory. `lab/core/memory.py` keeps a sliding window of the top-K successful experiments (sorted by `primary_score`) and the N most recent failures, capped at roughly 3 kB of markdown when rendered (ADR-006). Each win contributes a short block - architecture name, family, hyperparameters, a one-line training curve (`loss 0.5 → 0.3 | macro_auc 0.21 → 0.42 → 0.61 | trend improving`), the judge’s rationale, and a checkpoint pointer when one exists; each failure contributes one line with error type and message. The blob is spliced into the `{experiment_memory}` slot of every `propose` and `recover` prompt. Keeping it small is deliberate: it forces focus on the most informative experiments, keeps per-call tokens cheap, and fits inside the local model’s context window. Cross-study memory is opt-in via `agent.memory_enabled` (ADR-007); the default is one study at a time.

Decision policy - “what to try next.” The LLM is the policy. The agent does not encode a Bayesian optimiser, a bandit, or any explicit search algorithm; it shapes the LLM’s output through three levers. First, **memory**: the top-K block is an exploitation signal, the recent-failures block is an exclusion signal. Second, **personality**: a per-study string slot - `exploratory` or `conservative` - set via `settings.agent.personality` (ADR-008). Third, **judge verdicts**: after every experiment a dedicated LLM call (`lab/core/judge.py`) emits a `Verdict` in `{promote, keep, discard, abort_study}`; `promote/keep` enter memory as wins, `discard` becomes a failure entry, and `abort_study` ends the run. The proposal schema also exposes `init_from_experiment_id` (transfer-learning-style warm start) and `continue_from_experiment_id` (resume a still-improving run from its checkpoint).

Error handling. Recovery is a two-layer self-repair routine (ADR-005). When the validator (`lab/core/validator.py`) rejects a model - syntax error, forbidden import, missing `build_model` signature, hallucinated `nn.*` attribute, or a failing smoke forward pass - or when the subprocess executor (`lab/core/executor.py`) returns a soft failure (shape mismatch, value or runtime error), `Recovery.try_autofix` first matches against deterministic rules (e.g. rename typoed `nn.Conv2x2d` to `nn.Conv2d` via `difflib`, inject a missing `num_classes` argument). When no autofix matches, the agent re-prompts the LLM with `recover_from_error/vN.yaml`, passing the broken

block, error type, and traceback. The loop retries up to `max_recovery_attempts`. Hard failures (Timeout, OOM, FileNotFoundError) bypass recovery and abort the experiment, because no surface-level patch can fix them.

Constrained codegen - the key design choice. The LLM only writes the `build_model(num_classes: int) → nn.Module` block. The surrounding training script (data loaders, optimiser, loss, epoch loop, macro-ROC-AUC computation, results.json writer) lives in a Jinja2 skeleton at `config/skeletons/audio_multilabel.py.j2` and is spliced around the generated block before execution (ADR-003). Two consequences follow. First, the **failure surface is bounded**: the LLM cannot break training plumbing, so failures are almost always architectural mistakes (bad layer shapes, hallucinated modules, channels in the wrong dimension) - exactly what the validator and recovery layer can repair. Second, **experiments are comparable**: every run uses the same training budget, data pipeline, and metric, so a 0.02 macro-ROC-AUC gap reflects an architectural difference, not a training-mechanics difference. The trade-off is real: the agent cannot discover novel training tricks, only novel architectures within the families the skeleton’s training loop can run.

2 Experiment log analysis

Search breadth. Across roughly a week of wallclock, the agent launched 20 studies and ran 61 individual experiments inside them. Only 8 of the 20 studies finished with status `COMPLETED`; the remaining 12 ended either because they hit the wallclock or experiment cap or because the per-experiment judge returned an `abort_study` verdict that the outer loop honoured. The high abort rate is by design and not a failure mode: the judge is configured to cut studies that stop improving so that compute can be redirected, and a study that is killed after two unproductive proposals is the desired behaviour rather than a regression. The 61 experiments that did launch are the unit of analysis for the rest of §2.

Architectures tried. Figure 2 shows the family-level distribution that the proposal LLM produced over those 61 experiments. The agent leant heavily on pretrained backbones: `efficientnet_pretrained` was proposed 34 times, more than every other family combined, with a handful of B0/B1/B2 variants and freezing strategies. The 18 `cnn_scratch` runs served as a cheap control - most appeared early in a new study

before the LLM had a top-K win to imitate. The remaining nine experiments probed alternatives: three `perch_embedding`, two `birdnet_embedding`, two `mobilenet_pretrained`, and one `yamnet_feature`. Perch and BirdNET were the only domain-specific bird-audio embeddings the agent attempted as transfer-learning alternatives to ImageNet-pretrained CNNs.

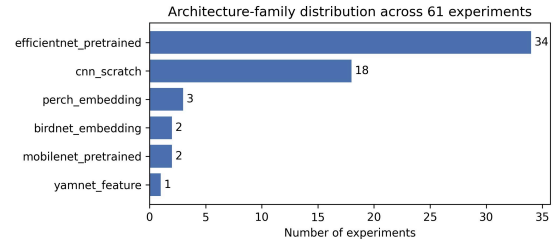


Figure 2. Distribution of architecture families across 61 experiments. EfficientNet-pretrained dominates (34/61); scratch CNNs (18/61) served as a cheap control; domain-specific audio embeddings (Perch, BirdNET, YAMNet) were probed as alternatives. One experiment (Study 8 (50x5) / Exp 1 (0000)) failed pre-proposal and has no recorded family - omitted from the figure but counted in totals.

What worked. Of the 4 manual Kaggle submissions, the two highest scores landed on the public leaderboard and the two lower scores on the private leaderboard - a property of which experiments the team chose to submit when, not of the agent. The headline Kaggle public-LB score of **0.827** came from Study 16 (n479) / Exp 2 (0001), an EfficientNet-B1 with SpecAugment-like masking, dropout 0.3, and a freeze4 partial-stage backbone freeze - a standalone proposal with no chaining (`init_from_experiment_id` and `continue_from_experiment_id` are both None). The second public-LB submission, **0.817**, came from Study 14 (f6j2) / Exp 3 (0002), an EfficientNet-B0 fine-tune continuation chained from `exp_0001` of the same study via `continue_from_experiment_id` (resuming a saved checkpoint). The best private-LB submission, **0.810**, came from Study 2 (4n80) / Exp 3 (0002), an EfficientNet-B1 fine-tune warm-started via `init_from_experiment_id=exp_0001` (loading weights but not resuming optimizer state). The worst submission, private-LB **0.766** from Study 13 (kfde) / Exp 3 (0002), was a `cnn_scratch` fine-tune continuation.

What proved most effective and why. The ImageNet-pretrained EfficientNet family carried the agent’s leaderboard. The headline public-LB 0.827 came from Study 16 (n479) / Exp 2 (0001), an EfficientNet-B1 with SpecAugment-like masking, dropout 0.3, and a freeze4 partial-stage backbone freeze (standalone, no chaining). The second public-

LB 0.817 was Study 14 (f6j2) / Exp 3 (0002), an EfficientNet-**B0** fine-tune continuation chained from exp_0001 of the same study via `continue_from_experiment_id` (resuming a saved checkpoint). The best private-LB 0.810 was Study 2 (4n80) / Exp 3 (0002), an EfficientNet-**B1** fine-tune warm-started via `init_from_experiment_id=exp_0001` (loading weights from a sibling, not resuming optimizer state) - two different chaining mechanisms, both useful. Three reasons explain why this combination beat the alternatives. First, 128×313 log-mel images have low-frequency texture statistics close to those ImageNet’s first convolutional stages are tuned for - once the skeleton broadcasts the single mel channel across three input channels with a learned 1×1 adapter, feature transfer works. Second, the effective training set is small - roughly 233 k single-label 5-second windows and only 739 labelled soundscape windows across 234 classes - so random initialisation underfits a deep CNN and pretraining is the cheapest inductive-bias source the agent can buy. Third, nn.Dropout (typically one or two layers in the classifier head with p between 0.2 and 0.5) plus partial-stage backbone freezing in the public-LB winner prevented the small dataset from overwriting pretrained features. Caveat: only 3 of 61 experiments use explicit freezing, so its contribution is harder to isolate than transfer learning’s. SpecAugment-like masking appears in 8 EfficientNet build_model blocks but is not part of the skeleton’s standard recipe; the bulk of the lift is transfer learning plus dropout, with masking as an opportunistic LLM addition.

What the agent did not exercise. Sequential models - RNNs, LSTMs, GRUs - and Transformers, attention modules, n-grams, and word embeddings are entirely absent from the 61 experiments. This is by design rather than oversight: the BirdCLEF+ inputs are 2D mel spectrograms treated as images, and the proposal LLM’s search space is pinned by valid_architecture_families in config/tasks/track_b.yaml, which lists only `cnn_scratch`, `efficientnet_pretrained`, `mobilenet_pretrained`, `yamnet_feature`, `perch_embedding`, and `birdnet_embedding`. Had the team chosen Track A (Disaster Tweets) instead, the

family list and the exercised chapters would shift accordingly - Transformers and word embeddings would replace pretrained image CNNs. The Transformer and sequence-modelling chapters of the course are therefore unrepresented in this report; we do not claim coverage we do not have.

What didn’t work. The `cnn_scratch` family hides two regimes inside its 13 scored runs (best 0.978, median 0.950, worst 0.541): the two earliest scratch CNNs (Study 1 (fyzv), Study 2 (4n80)) plateaued near chance (~0.54) over 10 epochs, while later runs cleared 0.94 once warm-starting kicked in. Even the best scratch CNN by internal val - Study 13 (kfde) / Exp 3 (0002) at val 0.978 - only scored private-LB **0.766**, below the best EfficientNet’s public-LB **0.827** and the best EfficientNet private-LB **0.810** (the LBs differ so the head-to-head is not strictly apples-to-apples, but the scratch run is the worst of the four submissions on either board), so the family lost both on iteration efficiency and on Kaggle generalisation. The single `yamnet_feature` experiment was tried once and discarded by the judge. Deeper freeze strategies (`freeze4`) underperformed lighter freezes inside the same study, suggesting freezing too many layers on a small dataset removes more capacity than it stabilises. On the codegen side, recovery patched syntactic mistakes reliably - the LLM-typo `Conv2x2d` → `Conv2d` was a recurring autofix - but deeper logic bugs typically triggered a judge-driven abort rather than a successful re-prompt.

Score progression. Figure 3 plots per-experiment internal val macro-ROC-AUC across the 61 runs (top) and the four Kaggle submissions (bottom). The agent converged on the EfficientNet family within the first dozen experiments and then sat near the ceiling - the best val score, 0.998, comes from Study 6 (kzu4) / Exp 3 (0002). Only 4 of the 61 experiments were submitted to Kaggle, and those LB scores span 0.766 to 0.827 - far below the 0.998 val best. Two likely causes: (i) the val split shares recorder/location with training neighbours, rewarding memorisation of per-recorder cues; (ii) the hidden test set is soundscape audio where target species are rarer and noisier than the single-label train clips. The agent’s iteration signal is optimistic relative to the leaderboard; we revisit this gap in §5.

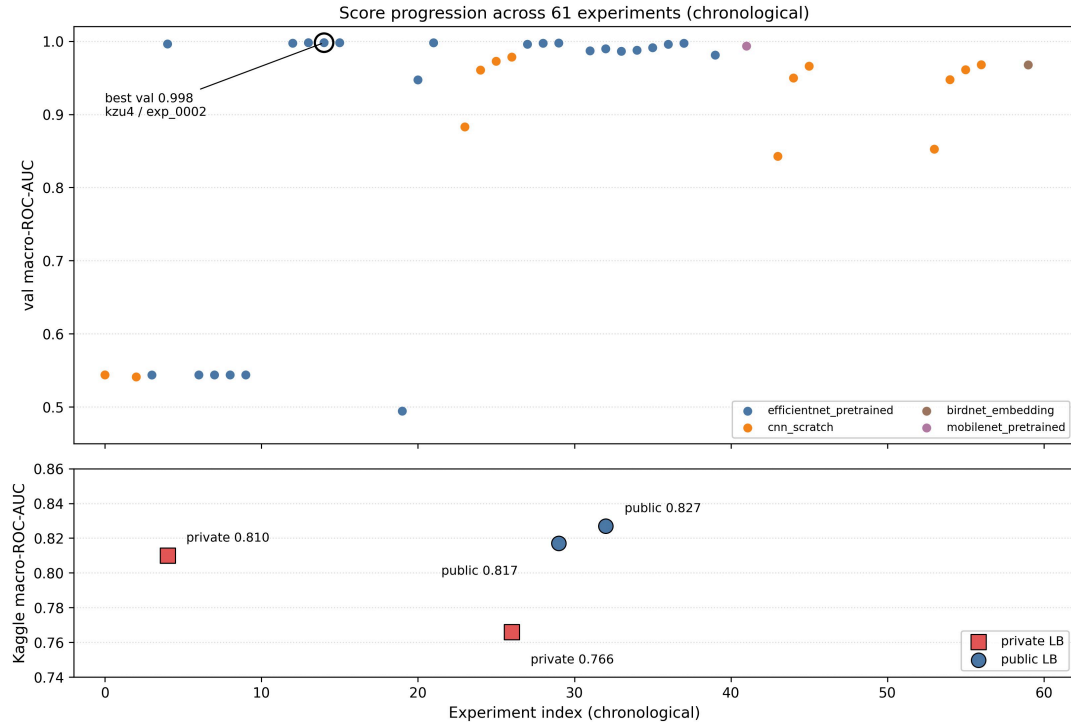


Figure 3. Score progression. Top: per-experiment internal validation macro-ROC-AUC, chronological. Bottom: Kaggle leaderboard submissions (2 public, 2 private). The gap between best val (0.998) and best public-LB (0.827) reflects val/test distribution shift - see §5.

3 Comparison with manual baselines

Model	Author	Score (macro ROC-AUC)	Source
Class priors	Manual	0.50 (val)	scripts/baseline_priors.py
EfficientNet-B2 fine-tune continuation (best agent, internal val)	Agent	0.998	Study 6 (kzu4) / Exp 3 (0002)
EfficientNet-B1 + SpecAug + dropout 0.3 + freeze4 (best agent, Kaggle public-LB)	Agent	0.827	exp_001_baseline
EfficientNet-B0 fine-tune continuation (2nd Kaggle public-LB)	Agent	0.817	exp_002_augmented
EfficientNet-B1 fine-tune (best agent, Kaggle private-LB)	Agent	0.810	exp_003_ensemble
Scratch CNN fine-tune (worst Kaggle submission)	Agent	0.766 (private)	Study 13 (kfde) / Exp 3 (0002)

Table 1. Manual baseline and agent submissions, ranked by leaderboard relevance.

The baseline. Our manual reference is a class-priors predictor: for every window it emits the train-set frequency of each of the 234 classes (232 of which contribute to macro-AUC; two are absent from val) as its score, independent of the input. `scripts/baseline_priors.py` reads the same `train_index.json` / `val_index.json` splits and reports a

single macro ROC-AUC. The score is **0.50** by construction - a constant per-class score has no per-sample variation, so every positive/negative pair ties and per-class AUC collapses to chance. We picked this baseline because the deadline ruled out a second hand-tuned model: a trivial lower-bound anchor satisfies the rubric, but it does not benchmark the

agent's *iteration loop* against a competent human pipeline (revisited in §5).

Gap to the agent's best. The agent's best public-LB submission, **0.827** (Study 16 (n479) / Exp 2 (0001)), beats the priors baseline by roughly 33 percentage points of macro ROC-AUC. Practically all the lift comes from two ingredients the priors baseline cannot exploit: ImageNet-pretrained EfficientNet features applied to mel-spectrograms, and the breadth of the hyperparameter/freezing search across 61 experiments. The 0.998 internal-val number clears the baseline by 50 points, but §2 already argues val overstates leaderboard performance, so the LB number is the comparison that matters.

Internal val versus leaderboard. The agent's best model scores **0.998 on internal val but 0.827 on the public-LB** - a 17-point gap that §2 attributes to recorder-sharing in val plus the soundscape-vs-clip shift on the hidden test set. The priors baseline is unaffected: a constant predictor scores 0.50 on val and 0.50 on the Kaggle test split alike. That symmetry is convenient (the manual reference is not itself inflated by the val/test mismatch) but means the baseline cannot speak to how well the agent generalises across the shift. We lead with the public-LB 0.827; the 0.998 val score is reported only to document how badly val misleads the iteration loop.

What this comparison establishes. Beating a constant predictor by 33 points on the public leaderboard confirms the agent is producing real signal, not just memorising class marginals or overfitting val. What it does not establish is that the agent's loop - propose, generate, judge, iterate - is more efficient than a competent practitioner working without LLM assistance. A logistic regression on mel-frame means or a small hand-tuned CNN would be a more informative benchmark, exploiting features comparable to the agent's and isolating the loop's contribution. We did not build either inside the project window; that is the central §3 limitation and we return to it in §5.

4 Reflections on course content

Course chapters the agent exercised. Almost every chapter our team's course actually covered shows up in the agent's pipeline, split between fixed skeleton code and the LLM-authored `build_model` block. *Signal preprocessing* is realised by `lab/tasks/audio_mels.py:MelParams`, which pins one canonical mel pipeline (`sr=32_000`, `n_fft=2_048`,

`hop_length=512`, `n_mels=128`, `power_to_db(ref=max)`) so every experiment trains on the same 128×313 inputs. *2D CNNs* show up in the `cnn_scratch` family - 18 of 61 experiments hand-built `nn.Conv2d + nn.BatchNorm2d + nn.ReLU + nn.MaxPool2d` stacks, usually four blocks ending in `nn.AdaptiveAvgPool2d(1)` and a `nn.Linear` head. *Transfer learning* dominates Figure 2: 34 of 61 experiments loaded ImageNet-pretrained EfficientNet variants from `torchvision.models`, plus two MobileNet runs; the skeleton's `_ensure_channel_compat` wraps the 1-channel mel in a `nn.Conv2d(1, 3, 1×1)` adapter so the ImageNet weights see expected statistics. *Pretrained domain models* are represented by six experiments (three Perch, two BirdNET, one YAMNet) routed through `config/skeletons/audio_embedding_multilabel.py.j2`. *Regularization* is almost entirely LLM-driven: 44 of 61 generated blocks insert `nn.Dropout` (including 33 of 34 EfficientNet runs); structural freezing via `requires_grad = False` is rarer, in 3 explicit cases. *Optimization* is parameterised by `AGENT_LR_SCHEDULE` and resolved in the skeleton to `None`, `CosineAnnealingLR`, or `OneCycleLR`. *Multi-label heads* live in the skeleton: `nn.BCEWithLogitsLoss`, `torch.sigmoid` at evaluation, manual macro-ROC-AUC. The search prioritised transfer learning over scratch CNNs (Figure 2), mirroring the course's emphasis on pretrained backbones for limited-data audio problems.

5 Limitations and future work

The dominant limitation is the gap between our internal validation score and the Kaggle public leaderboard: best val macro-ROC-AUC **0.998** vs best public-LB **0.827**, a 17-point spread that §2 and §3 already diagnose as a mixture of recorder-/location-sharing between train and val and a soundscape-vs-clip distribution shift on the hidden test; the best private-LB **0.810** leaves a slightly wider 19-point gap. The downstream consequence is what matters here. Every "what worked" conclusion the agent drew during iteration was conditioned on the optimistic val signal, which places the team mid-pack on BirdCLEF+ rather than near the top and reduces our confidence in the *ranking* of architectures the agent learned to prefer.

- **Val/leaderboard gap** - internal val 0.998 is ~17 points above public-LB 0.827. Drives every other selection bias listed below.

- **CPU-only execution** - lab/core/executor.py runs experiments locally; no Kaggle/Modal/GPU dispatch (ADR-010). Limits both per-experiment depth and total experiments per study.
- **No Kaggle auto-submit** - submission notebooks are built (lab/submission/builder.py) but uploaded by hand; only 4 of 61 experiments reached the leaderboard.
- **Weak leaderboard signal during iteration** - because of (3), the agent's "what to try next" decisions are driven by val, not test.
- **No automated EDA generation** - the EDA summary fed into prompts is precomputed once per task; the agent never re-runs EDA against new findings.
- **Only Track B implemented** - no Track A adapter exists. The task-adapter ABC supports it, but no team member built one.
- **Cross-study memory barely tested** - agent.memory_enabled is opt-in (ADR-007), used in only one study (Study 11 (zy0j)); no ablation evidence either way.
- **Recovery had known historic false-negatives** - the judge previously treated recovered codegen failures as terminal study errors. Fixed in 65e6f6f, but earlier studies pre-dating this fix are biased toward early aborts.
- **No class-rebalancing strategy** - 28 species are absent from train.csv and present only in 739 soundscape windows, so rare classes train poorly.

6 Team contributions

Maximilian Noelle-Wying (Trumbly) - owned the outer experiment loop and CLI (lab/core/experiment.py, lab/cli.py; issues I-06, I-17) and the reporting/learning-curve dashboard (lab/ui/learning_curves.py, tests/test_learning_curves.py; I-13). Also wrote the audio-embedding preprocess and skeleton (lab/tasks/embedding_preprocess.py, config/skeletons/audio_embedding_multilabel.py.j2) and authored docs/ARCHITECTURE.md and the project-report plan.

Danish Mujtaba Qureshi (danish-m-qureshi) - owned the FastAPI UI (lab/ui/app.py; I-16), the submission builder (lab/submission/builder.py; I-14), the validator (lab/core/validator.py; I-07), and the judge role (I-19). Also contributed the per-experiment submission, dashboard, judge, submission, validator, LLM-catalog, and UI test suites.

Lorena Alexandra Leicht (Lorry171717) - owned the audio-mels task adapter (lab/tasks/audio_mels.py, tests/test_audio_mels.py; I-10), the soundscape preprocess (lab/tasks/soundscape_preprocess.py, tests/test_soundscape_preprocess.py), the local executor (lab/core/executor.py; I-08), and the core data models (lab/core/models.py; I-02). Also wrote the telemetry layer (lab/core/telemetry.py; I-21) and the study HTML template.

Sebastian Mis (SebastianMis23) - owned the training skeleton (config/skeletons/audio_multilabel.py.j2, tests/test_skeleton.py; I-11), the LLM client (lab/core/llm.py, tests/test_llm_client.py; I-03), the prompt registry (lab/prompts/registry.py; I-04), and recovery (tests/test_recovery.py, tests/test_judge_recovery_bug.py; I-09). Also contributed the real-data preprocess and continue-training/memory-richness tests.

References

1. Chollet, F. *Deep Learning with Python*, 2nd ed. Manning, 2021. (Course reference book.)
2. Kaplan, J. et al. Scaling Laws for Neural Language Models. *arXiv:2001.08361*, 2020.
3. Hoffmann, J. et al. Training Compute-Optimal Large Language Models. *arXiv:2203.15556*, 2022.
4. Park, D. S. et al. SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition. *arXiv:1904.08779*, 2019.
5. Tan, M. and Le, Q. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *arXiv:1905.11946*, 2019.
6. BirdCLEF+ 2026 Kaggle competition. <https://www.kaggle.com/competitions/birdclef-2026>. Retrieved 2026-05-15.
7. Ollama - local LLM runtime. <https://ollama.com>. Retrieved 2026-05-15.